

Security Audit

Spectrum (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	42
Conclusion	43
Our Methodology	44
Disclaimers	46
About Hashlock	47

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Spectrum team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Spectrum Staking is a non-custodial blockchain infrastructure provider that operates validator nodes for multiple Proof-of-Stake networks, enabling users to participate in staking while retaining full control of their assets. The project focuses on reliability, security, and high availability by maintaining professionally managed validator infrastructure with continuous monitoring, automation, and hardened systems. Spectrum Staking does not custody funds or manage wallets; staking and reward distribution occur directly at the protocol level, with the company acting solely as a technical operator that supports network decentralization and validator performance across several PoS ecosystems.

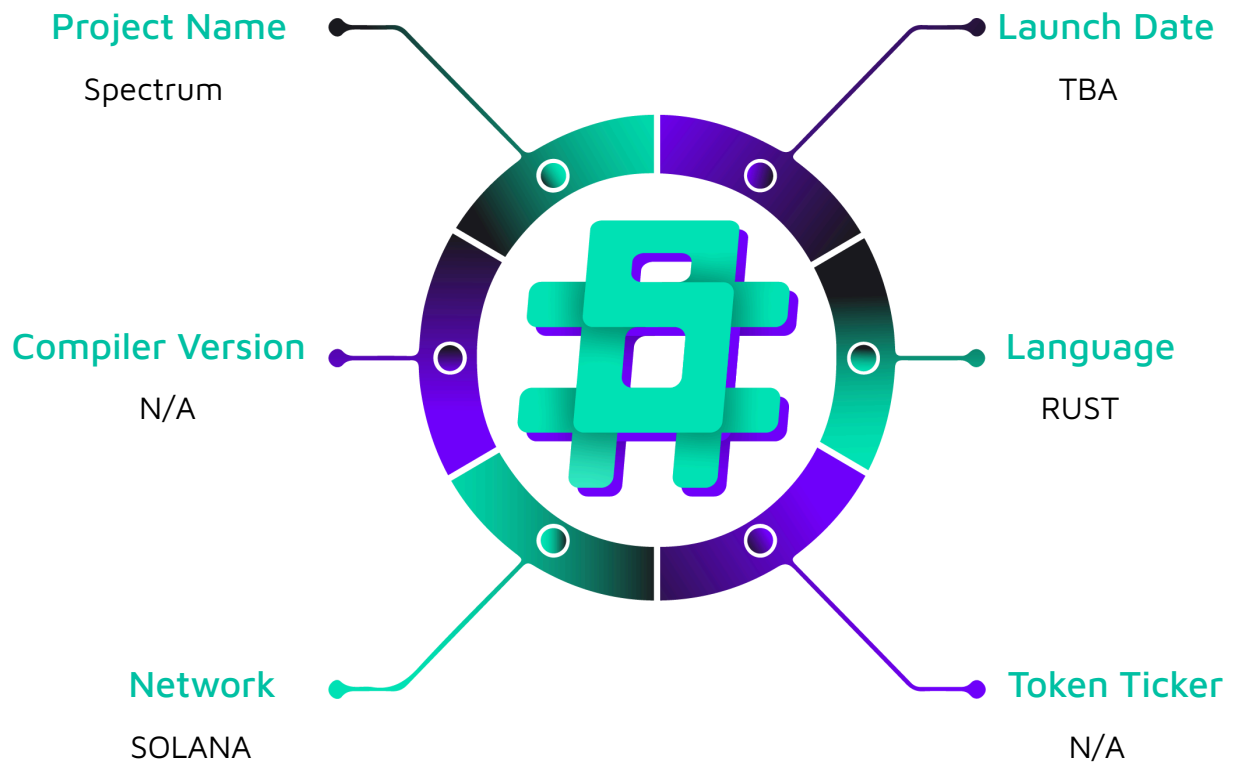
Project Name: Spectrum

Project Type: DeFi

Website: <https://spectrumstaking.net/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Spectrum project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Spectrum Smart Contracts
Platform	Solana / Rust
Audit Date	January, 2026
Audited Contract	lib.rs
Audited GitHub Commit Hash	ccc35b2a4d1c3489faa722a1fb645d3261f113ba
Fix Review GitHub Commit Hash	b40969b4d0344a6b61469213782c2a026e1564be

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High severity vulnerabilities

6 Medium severity vulnerabilities

10 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
multi_tier_staking: - Accept token deposits with optional lock periods and calculate shares based on tier thresholds for unlocked stakes or time-based multipliers for locked stakes - Track individual user stakes in a per-user PDA account, aggregating all unlocked deposits into a single stake at index zero while maintaining separate entries for each locked position - Distribute rewards proportionally across all stakers using a global accumulator pattern scaled by a precision factor, with rewards only claimable from unlocked stakes - Manage reward pool lifecycle, including funding, reward period configuration, pause or unpause controls, emergency stop, and post-period dust withdrawal - Handle withdrawals with lock period enforcement, partial withdrawal support, share recalculation, and automatic reward claiming - Provide emergency withdrawal, allowing users to retrieve principal from all stakes regardless of lock status	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Spectrum project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Spectrum project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] lib.rs - Unclaimed rewards are permanently lost when the claim transfer fails during withdrawal

Vulnerability Details

The `withdraw` function invokes `claim_rewards_with_transfer` but explicitly discards its return value, ignoring any transfer failures. Inside `claim_rewards_with_transfer`, the `stake.unclaimed_rewards` field is set to zero before the actual token transfer is attempted.

When the transfer subsequently fails due to an empty reward vault, insufficient funds, or token program errors, the transaction still succeeds because the error was discarded. The user's `unclaimed_rewards` have already been zeroed in the state, meaning those rewards are permanently lost.

This pattern of clearing accounting state before confirming the transfer violates the check-effects-interactions principle and creates an irrecoverable fund loss scenario.

```
// Always update unclaimed rewards and try to claim any that are available

// If claiming fails (e.g., empty reward vault), continue with withdrawal anyway
update_unclaimed_rewards(program_state, user_stakes, current_time)?;

let _ = claim_rewards_with_transfer(
    program_state,
    user_stakes,
    current_time,
    &ctx.accounts.reward_vault,
    &ctx.accounts.user_reward_token,
```

```
&ctx.accounts.reward_mint,  
  
&ctx.accounts.reward_token_program,  
  
);
```

Impact

Users will permanently lose accrued rewards whenever partial withdrawals coincide with reward vault insufficiency, which becomes increasingly likely as the reward period progresses and the vault depletes.

Recommendation

Propagate the `claim_rewards_with_transfer` error to fail the entire transaction, or restructure the function to only clear `unclaimed_rewards` after the transfer succeeds.

Status

Resolved

[H-02] lib.rs - Emergency withdraw bypasses lock periods and preserves claimable rewards

Vulnerability Details

The `emergency_withdraw_all` function allows users to withdraw their entire staked principal without checking whether individual stakes have completed their lock periods. The function iterates through all stakes, including those with active locks, and calculates pending rewards for every stake regardless of lock status. These accumulated rewards are then preserved in a newly created dummy stake with `lock_duration` set to zero, making them immediately claimable through the standard claim function.

This completely defeats the economic model where users accept illiquidity in exchange for higher multipliers. A user can deposit with the maximum allowed lock duration to receive the highest possible share allocation, wait a few minutes, call `emergency_withdraw_all` to retrieve their principal, and then immediately claim the disproportionately high rewards their inflated shares accrued.

```
// Create a single "dummy" stake to hold unclaimed rewards

if total_unclaimed > 0 {

    let dummy_stake = Stake {

        amount: 0,

        shares: 0,

        deposit_time: current_time,

        lock_duration: 0,

        reward_debt: 0,

        unclaimed_rewards: total_unclaimed,

    };

    user_stakes.stakes.push(dummy_stake);
}
```

```
// Transfer all deposited tokens back to user (no rewards)

let deposit_vault_seeds = &[

    b"deposit_vault".as_ref(),

    &[program_state.deposit_vault_bump],

];

let signer_seeds = &[&deposit_vault_seeds[..]];
```

Impact

Malicious users can farm maximum lock multiplier benefits without honoring any lock commitment, systematically draining rewards that should flow to legitimate long-term stakers.

Recommendation

Either enforce lock period checks within `emergency_withdraw_all`, really forfeit all rewards by not creating the dummy stake, or apply a penalty that removes the lock-based multiplier benefits when exiting early.

Status

Resolved

[H-03] lib.rs - Emergency stop creates invalid state when called before reward period begins

Vulnerability Details

The `emergency_stop` function validates that `reward_end_time` is greater than zero and that `current_time` is less than `reward_end_time`, but it does not verify that `current_time` is greater than or equal to `reward_start_time`. When an administrator calls `emergency_stop` before the reward period has actually begun, the function sets `reward_end_time` to `current_time`, which may be less than `reward_start_time`.

This creates an invalid state where `reward_end_time` is earlier than `reward_start_time`. The `calculate_reward_per_second` function handles this by returning zero when `reward_end_time` is less than or equal to `reward_start_time`, meaning no rewards will ever be distributed regardless of how much funding exists in the vault.

The program enters a permanent broken state where staking operations may still appear to function but reward accumulation is completely disabled.

```
pub fn emergency_stop(ctx: Context<AdminAction>) -> Result<()> {

    let program_state = &mut ctx.accounts.program_state;

    let current_time = Clock::get()?.unix_timestamp as u64;

    require!(program_state.reward_end_time > 0, StakingError::NoRewardPeriodSet);

    require!(current_time < program_state.reward_end_time,
StakingError::RewardPeriodAlreadyEnded);

    let previous_end_time = program_state.reward_end_time;

    program_state.reward_end_time = current_time;

    update_rewards(program_state, current_time)?;
```

```
emit!(EmergencyStopEvent {  
    previous_end_time,  
    new_end_time: current_time,  
});  
  
Ok::<(),  
}
```

Impact

A single mistimed administrator action can permanently disable all reward distribution while leaving the funded reward vault inaccessible to users.

Recommendation

Add a validation check requiring `current_time` to be greater than or equal to `reward_start_time` before allowing `emergency_stop` execution.

Status

Resolved

Medium

[M-01] lib.rs - Administrator can drain user rewards through the `withdraw_dust` function

Vulnerability Details

The `withdraw_dust` function transfers the entire balance of the reward vault to the administrator after `reward_end_time`, plus thirty days have elapsed. The function contains no verification that all users have claimed their entitled rewards, no check that `total_shares` equals zero, and no accounting for outstanding reward liabilities. Any user who has not claimed within this window permanently loses their rewards because the vault will be empty.

This vulnerability compounds with Issue 1 because subsequent withdrawal attempts will trigger failed claims that zero out `unclaimed_rewards`, while the discarded error allows the transaction to succeed. The thirty-day buffer provides inadequate protection for users who may be unaware of this hard deadline or unable to interact with the contract during that period.

```
pub fn withdraw_dust(ctx: Context<WithdrawDust>) -> Result<()> {

    let program_state = &ctx.accounts.program_state;

    let current_time = Clock::get()?.unix_timestamp as u64;

    require!(program_state.reward_end_time > 0, StakingError::NoRewardPeriodSet);

    require!(

        current_time > program_state.reward_end_time.checked_add(30 *
DAY_IN_SECONDS).ok_or(StakingError::MathOverflow)?,

        StakingError::DustWithdrawTooEarly

    );
}
```

Impact

Users who fail to claim within thirty days of reward period completion will have their earned rewards confiscated by the administrator with no recourse.

Recommendation

Track total outstanding reward liabilities globally and only permit withdrawal of the surplus above that amount, or require `total_shares` to equal zero before allowing dust withdrawal.

Status

Resolved

[M-02] lib.rs - Partial withdrawal from locked stakes eliminates time-based multiplier bonus

Vulnerability Details

When a user partially withdraws from a locked stake after the lock period expires, the `withdraw` function removes the original stake and creates a new unlocked stake for the remaining amount by calling `deposit_internal` with `lock_seconds` set to zero. The new stake receives share calculations based solely on threshold-based bonuses rather than the original time-based multiplier.

A user who locked one thousand tokens for 365 days to receive a two-times multiplier, then withdraws one hundred tokens after expiry, finds their remaining nine hundred tokens converted to an unlocked stake with significantly reduced share allocation. The user loses the benefit they earned by committing to the original lock period despite having honored that commitment in full.

Impact

Users are penalized for partial withdrawals even after completing their lock commitment, creating unfair economic outcomes that discourage partial liquidity access.

Recommendation

Preserve the original multiplier benefits when calculating shares for the remaining amount after partial withdrawal from expired locked stakes.

Status

Resolved

[M-03] lib.rs - Inconsistent share calculation logic between deposit and withdrawal operations

Vulnerability Details

The program applies different methodologies when calculating shares for unlocked stakes depending on whether tokens are being deposited or withdrawn. During deposits to an existing stake at index zero, the multiplier is calculated based solely on the new deposit amount, and the resulting shares are added to existing shares. During withdrawals, the multiplier is recalculated based on the total remaining amount, and shares are replaced entirely.

This inconsistency means two users with identical final balances can have different share allocations depending on their transaction history. A user making multiple small deposits accumulates shares based on each individual deposit's threshold tier, while another user making a single large deposit or triggering a recalculation through withdrawal receives shares based on the total amount.

The threshold-based bonus system becomes exploitable or produces unfair outcomes depending on interaction patterns.

Impact

Users can gain unfair share advantages through specific transaction sequences, or conversely lose expected share allocations through seemingly equivalent operations.

Recommendation

Implement consistent share calculation by always computing shares based on the total stake amount for unlocked positions, adjusting global shares by the net difference.

Status

Resolved

[M-04] lib.rs - Emergency withdraw function bypasses contract pause protection

Vulnerability Details

The `deposit`, `withdraw`, and `claim` functions all validate that `is_paused` equals `false` before proceeding, preventing user interactions when an administrator has paused the contract. The `emergency_withdraw_all` function contains no such check and remains fully operational regardless of pause state. This creates a protection gap where the pause mechanism cannot prevent fund movements during an active security incident.

If an administrator pauses the contract while investigating suspicious activity, users can still extract their deposits through emergency withdrawal.

Combined with the lock bypass vulnerability, malicious actors can continue exploiting the system even after administrators attempt to halt operations.

```
pub fn emergency_withdraw_all(ctx: Context<EmergencyWithdrawAll>) -> Result<()> {

    let program_state = &mut ctx.accounts.program_state;

    let user_stakes = &mut ctx.accounts.user_stakes;

    let current_time = Clock::get()?.unix_timestamp as u64;

    require!(current_time >= program_state.reward_start_time,
StakingError::RewardsNotStarted);

    require!(!user_stakes.stakes.is_empty(), StakingError::NoStakesToWithdraw);

    let mut total_amount = 0u64;

    let mut total_shares_to_remove = 0u64;

    let mut total_unclaimed = 0u64;

    // Calculate totals and update unclaimed rewards

    update_rewards(program_state, current_time)?;
```

```
for stake in &mut user_stakes.stakes {
```

Impact

The pause mechanism fails to provide complete operational control during security incidents, limiting administrators' ability to freeze the system during active exploits.

Recommendation

Either add pause validation to `emergency_withdraw_all` or explicitly document that emergency withdrawals intentionally bypass pause as a user protection measure.

Status

Resolved

[M-05] lib.rs - Fee-on-transfer token compatibility causes vault insolvency

Vulnerability Details

The program uses `TokenInterface` to support both standard SPL tokens and Token-2022 tokens, but the accounting logic assumes transferred amounts equal received amounts. When a user deposits tokens with `TransferFeeConfig` enabled, the deposit vault receives the amount minus the transfer fee while `stake.amount` is credited with the full requested amount.

Similarly, `add_funding` increments `total_rewards` by the requested amount rather than the actual amount received in the reward vault. Over time, the sum of credited stake amounts exceeds actual vault holdings. Later withdrawals will fail when the vault lacks sufficient tokens to honor the credited amounts.

The program provides no mechanism to detect or handle this discrepancy.

Impact

Using fee-on-transfer tokens causes progressive vault insolvency where late withdrawers cannot retrieve their full entitled amounts.

Recommendation

Either restrict the program to standard SPL Token only by validating the token program address, or measure actual received amounts by comparing vault balances before and after transfers.

Status

Resolved

[M-06] lib.rs - Permissionless initialization enables authority takeover

Vulnerability Details

The `initialize` function can be called by any account and sets the caller as the permanent program authority. The only restriction is that the `program_state` PDA must not already exist. If there is any delay between program deployment and initialization, an attacker can monitor the blockchain for the new program deployment and submit an initialization transaction before the legitimate deployer.

The attacker becomes the authority with full control over pause operations, reward funding, reward period configuration, emergency stop, and dust withdrawal. There is no mechanism to transfer authority or recover from a malicious initialization. The legitimate deployer would need to redeploy the entire program with a new program ID.

```
/// Initialize the staking program with configuration parameters

pub fn initialize(
    ctx: Context<Initialize>,
    silver_threshold: u64,
    gold_threshold: u64,
    diamond_threshold: u64,
    silver_bonus: u64,
    gold_bonus: u64,
    diamond_bonus: u64,
    linear_scale_constant: u64,
    quadratic_scale_constant: u64,
    max_multiplier_numerator: u64,
) -> Result<()> {
    require!(silver_threshold < gold_threshold, StakingError::InvalidThresholds);
    require!(gold_threshold < diamond_threshold, StakingError::InvalidThresholds);
```

```
let program_state = &mut ctx.accounts.program_state;
```

Impact

Attackers can permanently hijack program control if initialization is not performed atomically with deployment.

Recommendation

Hardcode the expected authority address in the program or require initialization to include a signature from a predetermined deployer key.

Status

Resolved

Low

[L-01] lib.rs - Reward accumulator overflow can permanently disable all program operations

Vulnerability Details

The `update_rewards` function calculates `reward_per_share_increase` by multiplying accumulated rewards by `PRECISION` (one billion) and dividing by `total_shares`, performing this calculation in u128 arithmetic. The result is then converted to u64 using `try_from`, which fails if the value exceeds u64 maximum.

When `total_shares` is very small, such as a single share from a minimal deposit, and significant time passes without interactions, the calculated `reward_per_share_increase` can exceed u64 maximum. Once this overflow condition occurs, every subsequent call to `update_rewards` fails with `MathOverflow`.

Since `deposit`, `withdraw`, `claim`, and `emergency_withdraw_all` all invoke `update_rewards` internally, the entire program becomes permanently inoperable. User deposits remain locked in the deposit vault with no mechanism for recovery.

```
let reward_per_share_increase_u128 = rewards_u128

    .checked_mul(precision_u128).ok_or(StakingError::MathOverflow)?

    .checked_div(total_shares_u128).ok_or(StakingError::MathOverflow)?;

// Convert back to u64, ensuring it fits

let reward_per_share_increase = u64::try_from(reward_per_share_increase_u128)

    .map_err(|_| StakingError::MathOverflow)?;

program_state.acc_reward_per_share = program_state.acc_reward_per_share

    .checked_add(reward_per_share_increase).ok_or(StakingError::MathOverflow)?;
```

Recommendation

Store `acc_reward_per_share` and `reward_debt` as u128 types to accommodate the full range of possible values, or implement bounds checking at configuration time to ensure the worst-case calculation cannot overflow.

Status

Resolved

[L-02] lib.rs - Missing multiplier parameter validation enables zero share deposits

Vulnerability Details

The `initialize` function accepts multiplier configuration parameters, including threshold bonuses and scale constants, without validating that they produce meaningful share calculations for expected deposit amounts. The share calculation divides the product of amount and `multiplier_numerator` by `BASE_MULTIPLIER` squared, which equals one trillion.

For shares to be at least one, the numerator must equal or exceed one trillion. With typical multiplier values around 1.3 million (representing 1.3x), minimum viable deposits must exceed 769000 tokens. Deposits below this threshold result in zero shares, triggering the `AmountTooSmall` error.

Poor parameter choices during initialization can make the program unusable for normal deposit amounts without any indication of misconfiguration.

```
program_state.linear_scale_constant = linear_scale_constant;  
  
program_state.quadratic_scale_constant = quadratic_scale_constant;  
  
program_state.max_multiplier_numerator = max_multiplier_numerator
```

Recommendation

Add parameter validation during initialization to ensure the combination of multiplier values and expected deposit ranges produces valid nonzero share calculations.

Status

Resolved

[L-03] lib.rs - Token-2022 mint extensions can compromise vault security

Vulnerability Details

The program accepts any mint compatible with `TokenInterface` without validating mint extensions. Token-2022 mints can include extensions such as `PermanentDelegate`, which grants a designated address the ability to transfer tokens from any account holding that mint without requiring the account authority signature. If the deposit or reward mint has a `PermanentDelegate` configured, that delegate can transfer tokens directly from the program vaults at any time without program authorization.

Other dangerous extensions include `DefaultAccountState`, requiring frozen accounts, and `TransferHook`, allowing arbitrary code execution during transfers. The program has no mechanism to detect or reject mints with these potentially malicious configurations.

Recommendation

Validate mint extensions during initialization and reject mints with `PermanentDelegate`, dangerous `DefaultAccountState` configurations, or other security-compromising extensions.

Status

Resolved

[L-04] lib.rs - No authority transfer mechanism

Vulnerability Details

The authority field in `ProgramState` is set during initialization and cannot be modified thereafter. No function exists to propose or accept authority transfers. If the authority private key is lost, compromised, or needs to be rotated for operational security reasons, there is no recovery path. All administrative functions including `pause`, `unpause`, `emergency_stop`, `add_funding`, `remove_funding`, `set_reward_period`, and `withdraw_dust` become permanently inaccessible.

The program continues operating but cannot be administered or upgraded in response to changing requirements or security incidents.

```
function _harvestRewards(uint256 _pendingRewards) private {
```

Recommendation

Implement a two-step authority transfer process where the current authority proposes a new authority, and the proposed authority must accept the transfer.

Status

Resolved

[L-05] lib.rs - Stake index instability from swap-and-pop withdrawal pattern

Vulnerability Details

When withdrawing a locked stake that is not the last element in the stakes vector, the `withdraw` function copies the last stake into the withdrawn stake's position before removing the last element. This swap-and-pop pattern changes the indices of remaining stakes.

A user with stakes at indices zero through three who withdraws stake one will find that stake three now occupies index one. Any external systems, user interfaces, or integrating contracts that reference stakes by index will experience unexpected behavior.

Users must re-query their stake list after each withdrawal to determine current indices. Automated systems may attempt operations on incorrect stakes.

```
// Handle locked stakes (indices 1+)

    if remaining_amount == 0 {

        // Remove locked stake completely using swap and pop

        let last_index = user_stakes.stakes.len() - 1;

        if stake_id as usize != last_index {

                                user_stakes.stakes[stake_id as usize] =
user_stakes.stakes[last_index];

        }

        user_stakes.stakes.pop();

    } else {

        // For partial withdrawal from locked stakes, remove the old stake and
create a new unlocked stake

        let last_index = user_stakes.stakes.len() - 1;

        if stake_id as usize != last_index {

                                user_stakes.stakes[stake_id as usize] =
user_stakes.stakes[last_index];
```



```
    }

    user_stakes.stakes.pop();

    // Create a fresh unlocked stake for the remainder (this will update
    stake 0)

    deposit_internal(program_state, user_stakes, remaining_amount, 0,
    current_time)?;
}
```

Recommendation

Document this behavior comprehensively or implement stable stake identifiers that persist across withdrawals.

Status

Resolved

[L-06] lib.rs - No minimum lock duration validation

Vulnerability Details

The `deposit` function accepts any positive value for `lock_seconds` without enforcing a minimum duration. Users can create locked stakes with trivially short periods, such as one second, which provides nearly immediate liquidity, while the code architecture suggests locks should represent meaningful commitments.

The multiplier calculation converts `lock_seconds` to days using integer division by 86400, so any lock under one day produces zero `lock_days` and receives no time-based multiplier. However, locks of exactly one day or slightly longer provide multiplier benefits while imposing minimal actual lockup.

The lack of a meaningful minimum allows users to game the system with short locks that provide some multiplier benefit without substantial commitment.

Recommendation

Enforce a minimum lock duration, such as seven days to ensure locks represent meaningful commitments proportional to their multiplier benefits.

Status

Resolved

[L-07] lib.rs - The `withdraw_dust` function does not update program state

Vulnerability Details

The `withdraw_dust` function drains the entire reward vault balance but contains no logic to update the corresponding program state fields. The function validates timing conditions, retrieves the remaining balance, executes the transfer, and emits an event, but never modifies `is_funded` or `total_rewards`.

After execution, `is_funded` remains `true` and `total_rewards` retains its original value despite the reward vault being empty. Subsequent `deposit`, `withdraw`, and `claim` operations pass their `is_funded` validation checks because the flag still indicates funding exists, but then fail during actual token transfers against the empty vault.

```
pub fn withdraw_dust(ctx: Context<WithdrawDust>) -> Result<()> {

    let program_state = &ctx.accounts.program_state;

    let current_time = Clock::get()?.unix_timestamp as u64;

    require!(program_state.reward_end_time > 0, StakingError::NoRewardPeriodSet);
    require!(

        current_time > program_state.reward_end_time.checked_add(30 *
DAY_IN_SECONDS).ok_or(StakingError::MathOverflow)?,

        StakingError::DustWithdrawTooEarly
    );

    let remaining_balance = ctx.accounts.reward_vault.amount;
    require!(remaining_balance > 0, StakingError::NoDustToWithdraw);

    // Transfer remaining tokens to admin

    let reward_vault_seeds = &[

        b"reward_vault".as_ref(),
```

```
    &[program_state.reward_vault_bump],  
];
```

Recommendation

Add state update logic to `withdraw_dust` that sets `is_funded` to `false` and `total_rewards` to zero after the transfer completes, and mark `program_state` as mutable in the `WithdrawDust` account context.

Status

Resolved

[L-08] lib.rs - Off-by-one error in maximum stakes enforcement

Vulnerability Details

The `UserStakes` structure defines stakes as a vector with `max_len` set to `MAX_STAKES_PER_USER`, which equals twenty. For locked stake deposits, the code checks whether `real_stakes_count` is less than `MAX_STAKES_PER_USER`, where `real_stakes_count` excludes the placeholder at index zero.

This allows twenty real locked stakes in addition to the placeholder, resulting in twenty-one total entries when pushing a new locked stake. The vector push operation will fail at runtime when attempting to serialize an account exceeding its allocated space.

Users who reach nineteen locked stakes can create a twentieth, but the transaction fails during account serialization with an unexpected error rather than the `MaxStakesReached` error they would expect.

Recommendation

Either increase `max_len` to twenty-one to accommodate the placeholder plus twenty locked stakes, or check total vector length before pushing rather than `real_stakes_count`.

Status

Resolved

[L-09] lib.rs - Threshold bonus progression not validated

Vulnerability Details

The `initialize` function validates that threshold token requirements increase from silver to gold to diamond, but does not validate that corresponding bonus percentages follow the same progression. An administrator could configure `diamond_bonus` lower than `gold_bonus` or `silver_bonus` higher than `gold_bonus`, creating counterintuitive reward structures where higher deposit tiers receive lower multipliers.

While this does not create direct exploits, it allows configuration states that violate reasonable user expectations about tier progression.

```
pub fn initialize(
    ctx: Context<Initialize>,
    silver_threshold: u64,
    gold_threshold: u64,
    diamond_threshold: u64,
    silver_bonus: u64,
    gold_bonus: u64,
    diamond_bonus: u64,
    linear_scale_constant: u64,
    quadratic_scale_constant: u64,
    max_multiplier_numerator: u64,
) -> Result<()> {
    require!(silver_threshold < gold_threshold, StakingError::InvalidThresholds);
    require!(gold_threshold < diamond_threshold, StakingError::InvalidThresholds);

    let program_state = &mut ctx.accounts.program_state;
    program_state.authority = ctx.accounts.authority.key();
```

```
program_state.deposit_mint = ctx.accounts.deposit_mint.key();  
  
program_state.reward_mint = ctx.accounts.reward_mint.key();  
  
program_state.deposit_vault = ctx.accounts.deposit_vault.key();  
  
program_state.reward_vault = ctx.accounts.reward_vault.key();  
  
  
program_state.threshold_token_requirement_silver = silver_threshold;  
  
program_state.threshold_token_requirement_gold = gold_threshold;  
  
program_state.threshold_token_requirement_diamond = diamond_threshold;  
  
  
program_state.threshold_bonus_percentage_silver = silver_bonus;  
  
program_state.threshold_bonus_percentage_gold = gold_bonus;  
  
program_state.threshold_bonus_percentage_diamond = diamond_bonus;
```

Recommendation

Add validation ensuring `silver_bonus` is less than or equal to `gold_bonus` and `gold_bonus` is less than or equal to `diamond_bonus`.

Status

Resolved

[L-10] lib.rs - The UserStakes account cannot be closed

Vulnerability Details

The program provides no mechanism to close `UserStakes` accounts after users withdraw all their stakes. Even with an empty stakes vector, the account persists and its rent remains locked. Users who fully exit the protocol cannot reclaim the lamports paid for account rent during their first deposit.

Over the program lifetime, abandoned `UserStakes` accounts accumulate as permanent state bloat.

```
function _harvestRewards(uint256 _pendingRewards) private {
```

Recommendation

Implement a `close_account` function that validates if the stakes vector is empty and closes the `UserStakes` account, returning rent to the user.

Status

Resolved

Centralisation

The Spectrum project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Spectrum project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#hashlock.